

Comparison of Continuous CoT vs Default Compute
Histogram Distribution of Steps to Convergence

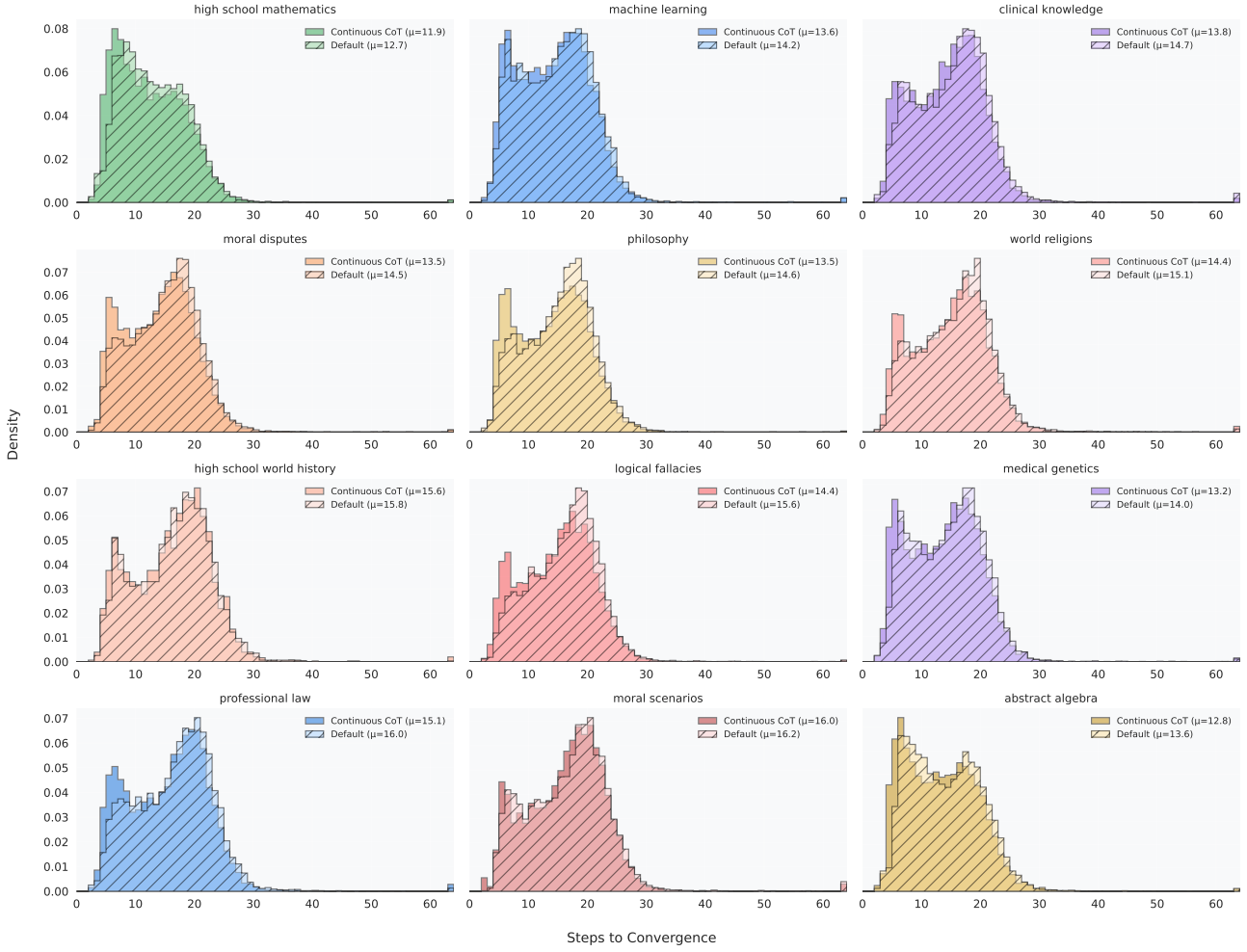


Figure 13: Additional categories for Figure 10 in the main body.

A. Additional Information

Potential Implications of This Work

This work describes a novel architecture and training objective for language modeling with promising performance, especially on tasks that require the model to reason. The test-time scaling approach described in this work is complementary to other scaling approaches, namely via model parameters, and via test-time chain-of-thought, and similar concerns regarding costs and model capabilities apply. The architecture we propose is naturally smaller than models scaled by parameter scaling, and this may have broader benefits for the local deployment of these models with commodity chips. Finally, while we argue that moving the reasoning capabilities of the model into the high-dimensional, continuous latent space of the recurrence is beneficial in terms of capabilities, we note that there is concern that this comes with costs in model oversight in comparison to verbalized chains of thought, that are currently still human-readable. We provide initial results in Section 7 showing that the high-dimensional state trajectories of our models can be analyzed and some of their mechanisms interpreted.

A.1. Classical Reasoning Problems

We include a small study of the classical problem of multi-operand arithmetic in Figure 14.

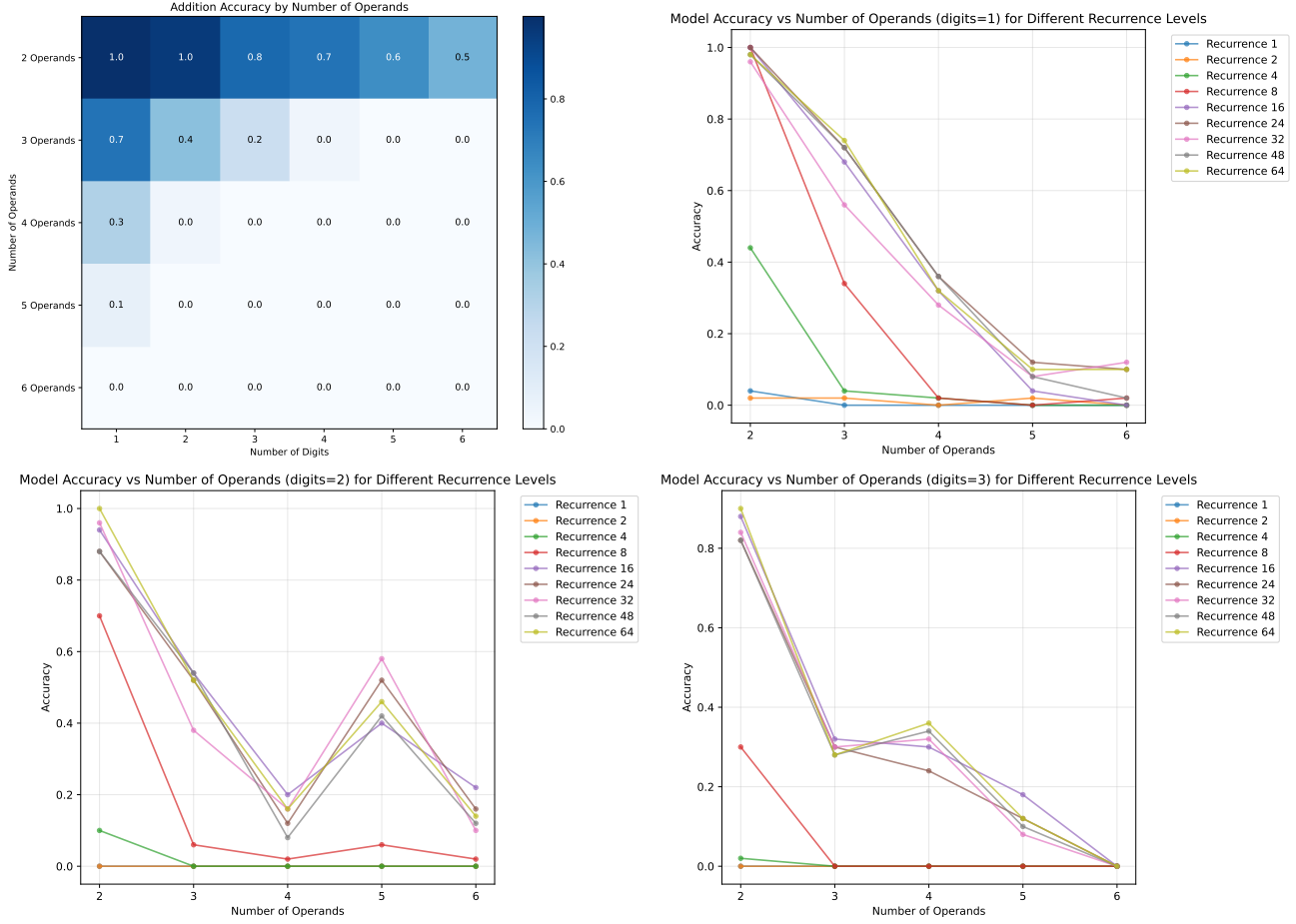


Figure 14: Multi-Operand Arithmetic. Following a precedent of training recurrent architectures for algorithmic and arithmetic tasks (Schwarzschild et al., 2021b; Bansal et al., 2022; Schwarzschild et al., 2023; McLeish et al., 2024), we explore whether our model can leverage increased test-time compute via recurrence to solve verbalized addition problems of increased difficulty. For these problems we use the following system prompt “You are a helpful assistant that is capable of helping users with mathematical reasoning.” embedded in a conversational chat template, and we present each problem by opening the first user turn of the conversation like so: `f“What is the result of ' + '.join(map(str, digits))?”` after randomly sampling numbers according to a certain operand count and digit count (base 10). We score correct answers by checking whether the correct sum appears as a string anywhere in the model’s output, and for each measurement, we average over 50 trials.

In the heatmap (top left), we evaluate the model at 32 recurrences to get an upper estimate of its addition performance at various difficulties. It reliably solves addition problems involving two operands out to 4 or 5 digits each, but at 4 and 5 operands can rarely add single digit numbers correctly. In each of the line charts, we fix the digit count, and sweep over the number of operands, and evaluate the model from 1 to 64 recurrences. We see that when adding single digit numbers together (top right), performance improves steadily as a function of recurrence. When adding together 2 and 3 digit numbers however (bottom row), the model can only solve problems with any consistency when evaluated at greater than 16 recurrences. Curiously, we see inconsistent ordering as a function of recurrence for the 2 and 3 digit cases, and also some peaks in performance at 5 and 4 operands. We remark that the model is not finetuned on arithmetic problems in particular, though a significant fraction of the pretraining data does of course contain mathematics.

Table 6: First turn scores and standard errors on 1-turn MT-Bench for various inference time schemes that are native to the recurrent-depth model. Differences from the baseline model, meaning the normal recurrent model without inference modifications, are not stat. significant.

Model	MT-Bench	Std. Error
cache compression, $s = 4$	5.856	0.395
baseline, 64 iterations	5.693	0.386
cache compression, $s = 16$	5.687	0.402
baseline, 32 iterations	5.662	0.388
cache compression, $s = 8$	5.631	0.384
KL exit, $t = 5 \times 10^{-4}$	5.562	0.389

A.2. Implementation Details

Device Speed Details Nominally, each MI250X (AMD, 2021) achieves 383 TFLOP in bfloat16, i.e. 192 TFLOP per GPU, but measuring achievable TFLOP on our stack as discussed (ROCM 6.2.0, PyTorch 2.6 pre-release 11/02) for arbitrary matrix multiplication shapes (i.e. we measure the peak achievable speed of the best possible shape iterating over shapes between 256 and 24576 in intervals of 256 and 110 (Bekman, 2023)), we measure a peak of 125 TFLOP/s on Frontier nodes. Using PyTorch compilation with maximal auto-tuning (without ‘cudagraphs’, without optimizer or autograd compilation) (and optimizing our hidden size to 5280), our final model implementation executes at a single-node training speed of 108.75 TFLOP/s, i.e. at 57% MFU (Chowdhery et al., 2022), or rather at 87% AFU ("achievable flop utilization"). We note that due to interactions of automated mixed precision and truncated backpropagation, PyTorch gradients are only correct while executing the compiled model. We further circumvent issues with the flash attention implementation shipped with PyTorch sdpa using the AMD fork of the original flash attention repository⁶, which can be found at <https://github.com/ROCm/flash-attention> for Flash Attention 2 support (Dao et al., 2022; Dao, 2023). We experiment with fused head and loss implementations⁷, but ultimately find that the most portable choice on our AMD setup is to let torch compilation handle this issue.

Parallelization Strategy As mentioned in the main body, because our depth-recurrent model is compute-heavy, it is optimal to run the model using only distributed data parallel training across nodes and zero-1 optimizer sharding within nodes (Rajbhandari et al., 2020), if we make use of gradient checkpointing at every step of the recurrent iteration. This allows us to eschew more communication-heavy parallelization strategies that would be required for models with the same FLOP footprint, but more parameters, which require substantial planning on this system (Singh et al., 2024; Singh and Bhatele, 2022). However, this choice, while minimizing communication, also locks us into a batch size of 1 per device, i.e. 4096 in total, and 16M tokens per step.

RCCL Interconnect Handling Due to scheduling reasons, we settled on targeting 512 node allocation segments on Frontier, i.e. 4096 GPUs. However, this posed a substantial network interconnect issue. The connection speed between frontier nodes is only acceptable, if RCCL (AMD GPU communication collectives) commands are routed through open fabrics interface calls, which happens via a particular plugin⁸. To achieve sufficient bus bandwidth above 100GB/s requires NCCL_NET_GDR_LEVEL=PHB, a setting that, on NVIDIA systems, allows packages to go through the CPU, and only uses direct interconnect if GPU and NIC are on the same (NUMA) node (Wu and Stock, 2024). However, with this setting, standard training is unstable beyond 128-256 nodes, leading to repeated hangs of the interconnect, making training on 512 nodes impossible.

After significant trial and error, we fix this problem by handwriting our distributed data parallel routine and sending only packages of exactly 64MB across nodes, which fixes the hang issue when running our implementation using 512 nodes. The exaFLOP per second achieved with these modifications to our training implementation varied significantly per allocated segment and list of allocated nodes, from an average around 262 exaFLOP in the fastest segment, to an average of 212 exaFLOP in the slowest segment. This is a range of 52-64 TFLOP/s per GPU, i.e. 41%-51% AFU, or 1-1.2M tokens per

⁶<https://github.com/Dao-AILab/flash-attention/>

⁷https://github.com/JonasGeiping/linear_cross_entropy_loss

⁸<https://github.com/ROCm/aws-ofi-rccl>